Software Requirements Specification

for

BetterDay

Version 1.0.0 approved

Prepared by,

Team F Fudge

Pub 3.13.26

Table of Contents

1. Introduction **3**
1.1 Purpose 3
1.2 Document Conventions 3
1.3 Intended Audience and Reading Suggestions 3
1.4 Product Scope 3
1.5 References 3
2. Overall Description **4**
2.1 Product Perspective 4
2.2 Product Functions 4
2.3 Assumptions and Dependencies 4
3. External Interface Requirements **5**
3.1 User Interfaces 5
3.2 Hardware Interfaces 11
3.3 Software Interfaces 11
4. Description and Priority **12**
4.1 Description 12
4.2 High Priority 12
5. Requirements **12**
5.1 Functional Requirements 12
5.2 Nonfunctional Requirements 13
5.3 Ethical and Legal Requirements 13
6. Appendices **14**

Revision History

Name	Date	Reason for Change	Version
Ben, Tom, Gavin, Will and Daniel	3/6/26	Initial setup	1.0
Ben, Tom, Gavin, Will and Daniel	3/13/26	Enhancements	1.01

1. Introduction

1.1 Purpose

The application betterDays is a web application that intakes user locally-stored data and reorganizes it into a easily readable UI. betterDays can make updates to the data, delete fields of the data, read the data and create new fields with the data. Where the user has the option to store it as a csv, or as a json document. The application comes with a FAQ and has several views. The application has the following views: Tasks, Calendar, FAQ and potentially more.

1.2 Document Conventions

This document used a unique convention described in the read me. The document also uses Pascal case nomenclature.

1.3 Intended Audience and Reading Suggestions

This document is intended to be viewed stake holders, developers, project managers designers and developers. An overall description of the product, External Interface Requirements, and additional requirements are discussed below.

1.4 Product Scope

The software simply distributes the integration with local files, no database required to save cost and reduce the amount of data transmitted across the internet. It's purpose is intended for users of all age groups in an effort to organize the target user's life. The software is designed to appeal to these groups by gamifying tasks and so much more.

1.5 References

Github Repo: https://github.com/COS420-S2025/TeamF.git
Figma Link:
https://www.figma.com/design/3LXr2yIJCKvBXUsQ8148ZF/Untitled?node-id=9-2&p=f&t=B6XjF0g88Qznymcl-0

---

2. Overall Description

2.1 Product Perspective

Betterdays is being developed for everybody who needs a calendar. This app combines calendar functionality with habit tracking.

2.2 Product Functions

Tasks

- Create tasks
- Read tasks
- Update tasks
- Delete tasks

Tags

- Create tags
- Assign tags to tasks
- Filter by tags
- View tags

Viewing habits

- View your most commonly missed habits

JSON use

- Import from JSON
- Export to JSON

Calendar views

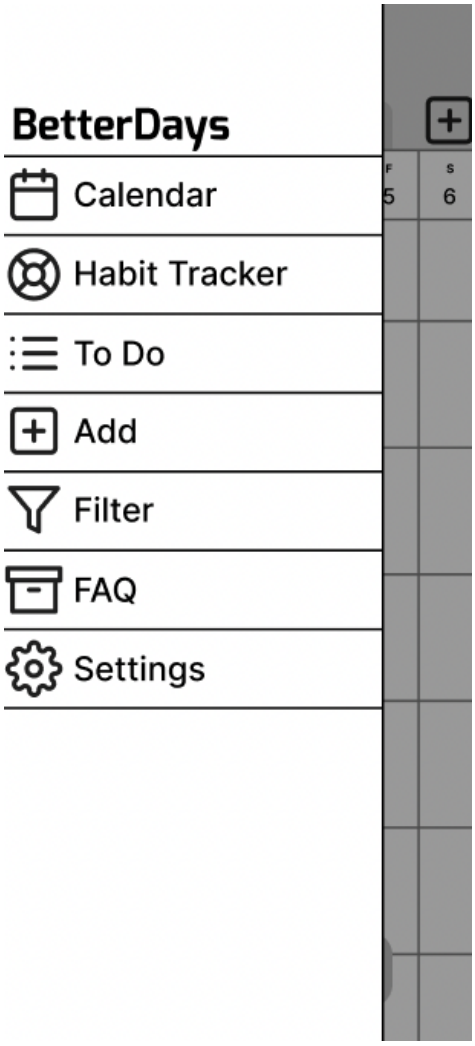
- Monthly view
- Weekly view
- Daily view

2.3 Assumptions and Dependencies

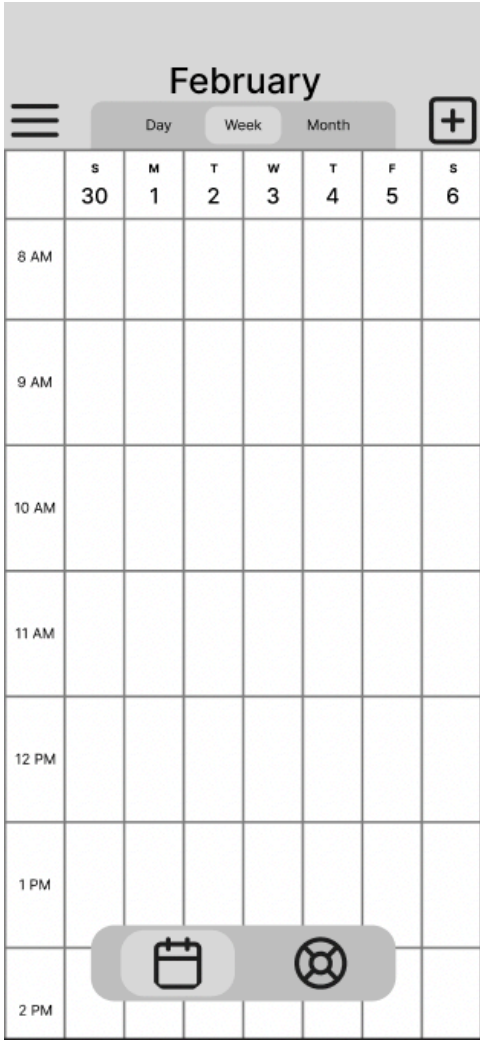
Betterdays is developed in React using Typescript. We are assuming users are not blocking Java Script, if they are, our application will not work on their device.

3. External Interface Requirements

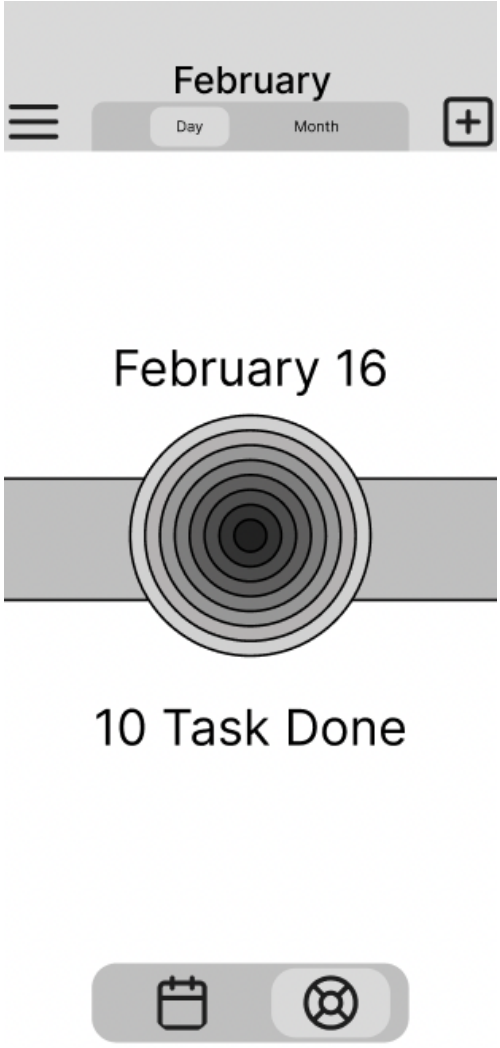
3.1 User Interfaces



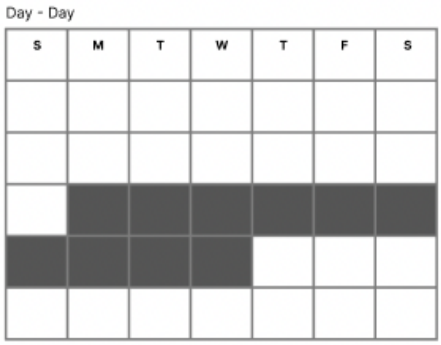
This is an example of the menu page allowing users to shift between the pages and tools without having clutter on the main pages. Clicking on any of the buttons will send you to a page or the respective tool.



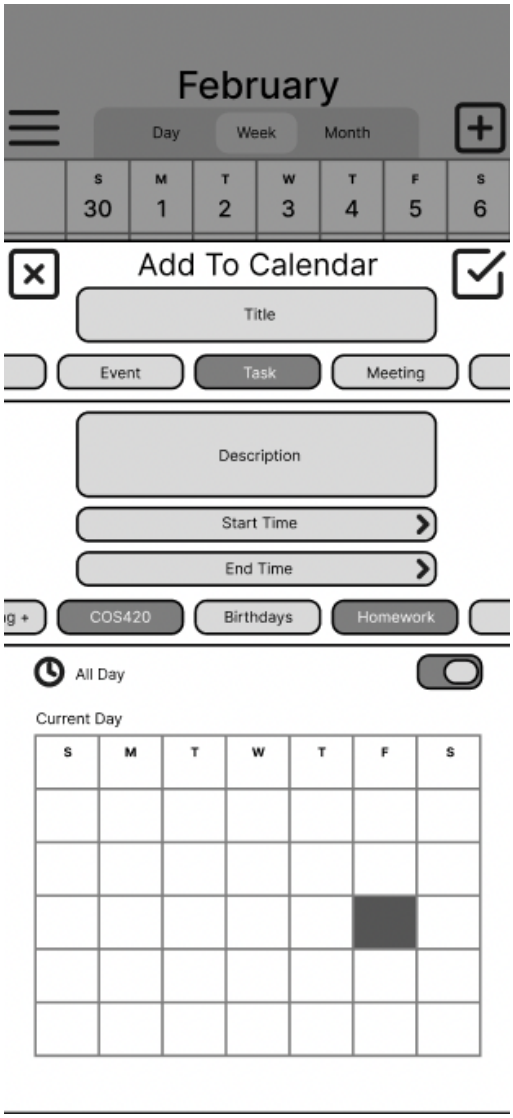
This is an example of the calendar in the weekly view. Clicking on the different views at the top will send you to different views, clicking add will send you to the create event page, clicking menu will send you to the menu, and clicking on the habit button on the bottom will send you to the habit view.



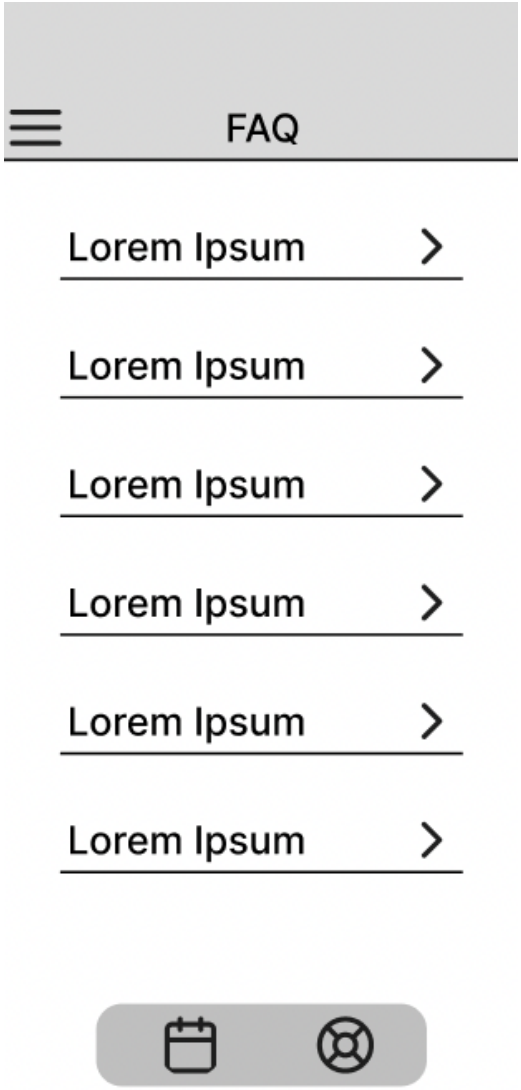
This is an example of the habit view. This view shows how many of your events and task you are completing in a day, giving the user a reason to continue using the app. The buttons at the bottom will switch between the 2 main pages of the app (calendar and habits), the buttons at the top will send you to the respective views for habit, the menu button will send you to the menu, and the add button will send you to the add event button.



This is the Filter Tool using this will allow the user to only see the task they want to see on their calender. Filling out the optional information you want to see and submitting the filter will show only the matching task.



This is the add tool you can choose to add calendar objects (ex. Events, Tasks, Meetings). You fill out the information needed for this event, and cancel it or submit the details and this will create a Calendar object.



This is the FAQ view. When the user has questions, they will go here to get answers. Clicking on a question will open the answer right under it.

This is a to-do section. A location for all the events, including those that may not have a time assigned to them and would not be on the calendar. Clicking on a date will drop down all events where you can check them off when done.

3.2 Hardware Interfaces

The software is designed to work on most browsers, because it is a browser based appication it can run on any operating system that can manage csv files and running a browser.

3.3 Software Interfaces

The software involved is simply the React basics and a calendar library because we don’t use a legitimate back end. The software requirements end with React.

4. Description and Priority

4.1 Description

The system shall have integrated calendars and dynamic task lists with filterable views to be able to both view high level summaries of their scheduling as well as be able to have a low level detailed task and event management system to handle complexities of any task.

4.2 Priority User Stories

1. As a User I want an FAQ page.
2. As a User I want to be able to hide other answers on the FAQ page.
3. As a user I want to see a month view of the calendar
4. As a user I want to see a daily calendar view
5. As a user I want to read calendar events
6. As a user I want to change the timeframe of the Calendar
7. As a user I want a task page.

5. Requirements

5.1 Functional Requirements

1. The system shall have a task section
2. The system shall allow the user to create events
3. The system shall allow users to delete events
4. The system shall allow users to create tasks
5. The system shall allow users to delete tasks
6. The system shall allow users to edit events
7. The system shall allow users to edit tasks
8. The system shall have a menu
9. The system shall have an interactive calendar
10. The system shall have optional linked data between the calendar and task list
11. The system shall have tags for events and tasks
12. The system shall toggle between views for task and Calendar: tasks and habits
13. The system shall let the user assign colors to tasks
14. The system shall let the user assign colors to events
15. The system shall have the ability to filter by multiple tags.
16. The system shall have a ‘popup’ menu and search bar to select tags to filter
17. The system shall have tasks filter by date
18. The system shall display a maximum of 5 events on a day in monthly calendar view
19. The system shall notify the user if task creation exceeds 2 seconds.

20. The system shall have a daily calendar view
21. The system shall have a weekly calendar view
22. The system shall have a monthly calendar view
23. The system shall notify the user of upcoming events and task deadlines
24. The system shall make the notification system optional
25. The system shall have a FAQ page.
26. The system shall have a habit section
27. The system shall have a monthly habit section
28. The system shall have front/back end using JSON

1. The system shall be able to hide FAQ answers

5.2 Non-functional Requirements:

1. The system shall upload the save-file document and fully populate the app within 45 seconds 90% of the time.
2. The system shall display newly created tasks in the UI within 1 second for 90% of operations.
3. The system shall notify the user if their task takes longer than 2 seconds to create after pressing submit.
4. The system shall save 2 years of calendar information to preserve efficiency
5. The system shall comply with WCAG 2.1 level AA.
6. The system shall provide visual confirmation for creating, editing, or removing a task/event within 2 seconds of event completion 90% of the time.
7. The system shall switch between page views in 3 seconds 95% of the time.
8. The system shall update habit stats within 30 seconds based on new calendar events added.
9. The system shall swap component states within 1 second 90% of the time.
10. The system shall send and receive data from the external database within 5 seconds 90% of the time
11. The system shall comply with UMaine academic integrity standards.

5.3 Ethical and Legal Requirements

1. The system shall comply with UMaine academic honesty standards.

6. Appendices